GO PROGRAMMING MASTER COURSE

Week 9, Day 1

Errors, wrapping, validation, slog — and a light register/login preview. Still no Postgres.

Umesh Indrajith · BSc Eng (Hons), University of Moratuwa
Senior Software Engineer / Lecturer

SINCE LAST WEEK

Repo owns the map. Handlers own HTTP status.

- Create / list / get still green through `StudentRepository`.
- Cold-call: what does `ok == false` mean in GetByID? Why not panic?
- Today we replace bool with real `error` values — and log properly.
- JWT middleware can wait; prefer register/login without locking student routes.

TODAY

If Day 1 of week 9 works, you can

- Treat `error` as a value; use a sentinel `ErrNotFound`.
- Wrap with `%w`; check with `errors.Is` / `errors.As`.
- Never panic in an HTTP handler — return `{"error":"..."}`.
- Validate with Gin `binding`; log 5xx with `log/slog`.
- Load config from env (`.env.example`, never commit secrets).

SOFTWARE ENGINEERING

Errors are part of the API contract

Clients and operators need predictable JSON and logs. A panic is a bug, not a status code.

- Same shape everywhere: `{"error":"student not found"}`.
- 4xx = client problem. 5xx = our problem — and we log it.
- Secrets live in environment variables, not in git.

ERROR AS A VALUE

The second return is the handshake

```
s, err := repo.GetByID(id)
if err ≠ nil {
    // decide: 404, 400, or
    return
}
c.JSON(http.StatusOK, s)
```

- Ignoring `err` with _ needs a spoken reason.
- `error` is an interface: anything with `Error() string`.

SENTINEL VS CUSTOM

Name the expected failures

```
var ErrNotFound = errors.New("not

// later, richer:
type ValidationError struct { Fiel
func (e ValidationError) Error() st
```

- Sentinel: package-level `var ErrX = errors.New(...)` — compare with `errors.Is`.
- Custom type: when you need fields; unwrap with `errors.As`.
- Lab uses a sentinel `ErrNotFound`.

WRAPPING

%w, Is, and As

```
if err ≠ nil {
    return fmt.Errorf("get student
}
if errors.Is(err, apperrors.ErrNot
    c.JSON(404, gin.H{"error": "st
    return
}
```

- %w preserves the chain so callers can still Is the root cause.
- Do not wrap away detail you need in logs — add context, keep the cause.

HTTP MAPPING

Repo returns errors · handler picks status

- `errors.Is(err, ErrNotFound)` → **404**
- Bad input / bind failure → **400**
- Anything unexpected → **500** + `slog.Error`
- Never `panic(err)` inside a Gin handler.

VALIDATION

Reject junk before it hits the repo

```
type CreateStudentRequest struc
    FirstName string `json:"fir
    Email     string `json:"ema
    Course    string `json:"cou
}
if err := c.ShouldBindJSON(&req
    c.JSON(400, gin.H{"error":
    return
}
```

- Match this repo's `RegisterRequest` / `CreateStudentRequest` style.
- Clear `error` string beats dumping raw validator text (optional polish).

SLOG

Structured logs, not `fmt.Println`

```
slog.Info("student created",
slog.Error("create failed", "
```

- JSON handler in production: level, msg, attrs.
- Log 5xx with enough context to replay — never log passwords.
- Homework: kill leftover print debugging.

CONFIG

Environment, not hard-coded secrets

```
// internal/config – sha
type Config struct {
    Port      string
    JWTSecret string
}
// godotenv.Load() optio
// ship .env.example – n
```

- DB vars can appear in the struct now; we still do not connect Postgres this week.
- Default port `:8080` if empty.

AUTH · LIGHT

Hash passwords. Sign a string later.

- `bcrypt`: store a hash, never the password.
- JWT: a signed claim string — middleware can wait until week 15 if slow.
- Prefer `POST /api/register` (+ login) **without** locking student routes yet.

JWT PREVIEW

One slide — then stop

```
// idea only: header.payload.si
// payload might carry user_id
// verify signature with JWT_SE
```

Product story: login returns a token; later middleware checks it.

Do not protect `/api/students` today unless the room is flying.

SMS MILESTONE

Consistent errors + logs

```
GET /api/students/:id missing → 4
Unexpected repo failure → 500 + s
Optional: POST /api/register (in-
```

- Files to teach toward: `auth_handler.go`, `models/user.go`, `config/config.go`.
- Still no Postgres. Restart still clears memory.

LIVE DEMO

ERRNOTFOUND

Sentinel from repo → 404 in handler

```
// repository
if !ok { return models.Student{}, apperr

// handler
if errors.Is(err, apperrors.ErrNotFound)
    c.JSON(404, gin.H{"error": "student
    return
}
slog.Error("get student", "id", id, "err
c.JSON(500, gin.H{"error": "internal err
```

LIVE DEMO

SLOG + CONFIG

JSON logs and .env.example

```
slog.SetDefault(slog.New(slog.NewJSONHandler(os
cfg := config.Load()
r.Run(":" + cfg.Port) // default 8080

# .env.example
PORT=8080
JWT_SECRET=change-me
```

LIVE DEMO · STRETCH

REGISTER

POST /api/register in memory

```
// bcrypt hash → store User (pa
// conflict on duplicate userna
// Do not protect student route
```

Skip live if ErrNotFound lab needs the rest of class.

LAB · REST OF CLASS

Done when all of this is true

- `ErrNotFound` defined; repo returns it for missing students
- Handler maps not-found → 404; unexpected → 500 + slog
- Bind failures → 400 with `{"error":...}`
- No panics on the student paths
- Health still green; PR started

HOMEWORK

slog + .env.example

- Replace `fmt.Println` debugging with `slog` (Info/Error + attrs).
- Add `.env.example` (PORT, JWT_SECRET placeholders). Keep `.env` gitignored.
- Wire `internal/config` for port (and JWT secret for later).
- Stretch: in-memory register (+ login token if ready).

NEXT SESSION

Week 10 — goroutines and the scheduler

- Concurrency vs parallelism; `go f()`.
- Why handlers must not fire-and-forget without a plan.
- Bring clean error mapping and slog.

WORDS TO KEEP

Say these correctly next week

Sentinel

Named error value; compare with errors.Is.

Wrap

fmt.Errorf with %w keeps the cause.

slog

Structured logs with level + attrs.

Config

Secrets from env; example file only in git.

LAYOUT REMINDER

Where the new files live

```
backend/
  .env.example
  cmd/server/main.go
  internal/config/config.go
  internal/apperrors/errors.go      # ErrNotFou
  internal/models/{student,user}.go
  internal/repository/ ...
  internal/handlers/{student,auth}.go
```

HONEST LIMITS

What this week is not

- Not Postgres — users/students still in memory.
- Not full auth middleware on every route.
- Not a logging platform — stdout JSON is enough.
- Not panicking “to be honest” in handlers.

SAFETY

Secrets and passwords

- Never commit `.env` or real JWT secrets.
- Never log raw passwords or full tokens.
- `gofmt`; no binaries in git.

REVIEW BAR

Instructor merges when

- Missing student → 404 JSON, not panic / empty 200
- Unexpected errors → 500 + slog (evidence in PR)
- `.env.example` present; `.env` absent from git
- gofmt clean; PR describes error mapping

Questions